

Bit Manipulation

BINARY STRUCTURE

Powers of 2	1, 2, 4, 8, 16, 32, 64, 128...	exactly one 1 bit in binary
Powers of 2 - 1	1, 3, 7, 15, 31, 63, 127, 255...	every bit is 1 up to the MSB

BITWISE OPERATORS

OR (|) — Union

always goes up or equal

A	B	A B
0	0	0
0	1	1
1	0	1
1	1	1

- Like set union — includes bits from either operand
- No information loss
- Not invertible — can't recover originals
- Result is always $\geq$ both inputs

AND (&) — Intersection

always goes down or equal

A	B	A & B
0	0	0
0	1	0
1	0	0
1	1	1

- Like set intersection — only bits set in both
- Information loss — bits are cleared permanently
- Not invertible
- Result is always $\leq \min(a, b)$

XOR (^) — Symmetric Difference / Parity

unmatched bits -> 1

A	B	A ^ B
0	0	0
0	1	1
1	0	1
1	1	0

- Bits that differ -> 1, bits that match -> 0
- Cancels duplicates — great for finding unique elements
- Addition without carry
- Self-inverse: $a \wedge a = 0$
- Great for toggling bits

SHIFTS

<code>x << k</code>	Left shift — multiplies by 2^k
<code>x >> k</code>	Right shift — divides by 2^k

BIT MANIPULATION TRICKS

Set bit k to 1	Clear bit k to 0
<code>x = (1 << k)</code>	<code>x &= ~(1 << k)</code>
Toggle bit k	Check if bit k is set
<code>x ^= (1 << k)</code>	<code>if (x & (1 << k))</code>

COUNTING BITS (POPCOUNT)

Manual loop	Built-in functions
<pre>while (n) { count += (n & 1); n >>= 1; }</pre>	<code>__builtin_popcount(x)</code> — for int <code>__builtin_popcountll(x)</code> — for long long

COMMON PATTERNS

Remove lowest set bit	<code>n &= (n - 1)</code>
Get lowest set bit	<code>low = n & -n</code>
Clear lowest set bit	<code>n = n & (n - 1)</code>
Check power of 2	<code>n > 0 && (n & (n-1)) == 0</code>
Turn off rightmost bit	<code>x &= x - 1</code>
Keep only rightmost bit	<code>x & -x</code>

Swap two numbers using XOR

<code>a = a ^ b</code>	<code>b = a ^ b</code>	<code>a = a ^ b</code>
------------------------	------------------------	------------------------

KEY PROPERTIES TO REMEMBER

OR ()	Result always $\geq$ both inputs. Bits only go up.
AND (&)	Result always $\leq \min(a, b)$. Bits only go down.
XOR (^)	Counts unmatched bits. Like addition without carry. Self-inverse: $a \oplus a = 0$